

UNIVERSIDAD CATÓLICA ANDRÉS BELLO

FACULTAD DE INGENIERÍA

ESCUELA DE INFORMÁTICA

Simulación de Kernel y Gestor de Procesos en Consola

Bobes, Sebastián
Yajure, Simón
Aponte, Daniela

Tutor:
Castro Marcel

Caracas, 11 de noviembre de 2026

Estructura del Código

Aunque físicamente el código se encuentra contenido en un único archivo fuente (main.cpp), su diseño interno se encuentra lógicamente segregado en cuatro modulo funcionales bien diferenciados, facilitando su compresión, mantenimiento y una futura refactorización.

Módulo Lógico	**Componentes / Funciones Asociadas**	**Responsabilidad Principal**
1. Utilidades y Análisis Léxico	state_name, parse_integer, trim, format_seconds, tokenize, TraceProcess	Procesamiento de cadenas de texto, conversión segura de tipos (std::from_chars), análisis léxico (*lexer*) de comandos y formateo de marcas temporales.
2. Núcleo de Planificación y Procesos (Simulator**)**	create_process, load_trace, print_processes, print_pcb, set_scheduler, tick_locked, take_ready_locked	Gestión del ciclo de vida de los procesos (PCB), mantenimiento de la cola de listos (ready_queue), aplicación de algoritmos de planificación (**Round Robin** y **SRTF**) y cálculo de métricas de rendimiento.
3. Concurrencia y Sincronización	start_worker, shutdown_and_join, cpu_loop, MutexGuard, variables POSIX	Control del hilo independiente de la CPU en segundo plano, exclusión mutua de datos compartidos y temporización de los ticks de reloj mediante primitivas POSIX.
4. Interfaz de Línea de Comandos (CLI**)**	run_cli, print_help, require_count, main	Interacción en tiempo de ejecución con el usuario, lectura y validación sintáctica/semántica de comandos ingresados por consola.

Declaración Obligatorio del Uso de Inteligencia Artificial

Se deja constancia explícita de que para la elaboración, estructuración y análisis detallado de este proyecto se ha empleado asistencia de Inteligencia Artificial.

La IA fue utilizada exclusivamente como un recurso de apoyo pedagógico y de ingeniería de software para auditar el código fuente y clarificar los mecanismos de concurrencia y diseño modular.

La autoría, validación, comprensión conceptual y aplicación del código implementado corresponden en su totalidad al equipo de desarrollo del proyecto.

Uso Ético de la inteligencia Artificial

- *Inteligencia Artificial usada*: Gemini 3 Pro y Gemini 3.8 Flash
- *Prompts principales*:
  Cita 1:

  "Estoy intentando depurar el bucle del hilo de la CPU (cpu_loop) que utiliza pthread_cond_timedwait. A veces, cuando pauso y reanudo la simulación rápidamente, el hilo principal parece no despertar al hilo de trabajo o se queda bloqueado esperando la señal. Ayudame a pensar que fallas pueden estar ocurriendo?"

  Cita 2:

  "Actúa como un tutor socrático. Estoy diseñando la lógica de desalojo para el algoritmo SRTF (Shortest Remaining Time First) cuando llega un proceso con menor ráfaga que el actual, pero no quiero que me des la solución escrita. Hazme preguntas orientadoras para ayudarme a desarrollar la evaluación de las colas y decidir cuando se interrumpe le proceso"